

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

INF239 SISTEMAS OPERATIVOS

Semestre 2025-2

Laboratorio 4

1).- (8 puntos) Se tiene el siguiente programa en *Golang*:

```
1 // varcond.go
2 package main
3
4 import (
5     "fmt"
6 )
7
8 func child() {
9     fmt.Println("Hijo")
10 }
11
12 func main() {
13     fmt.Println("Padre: inicio")
14     go child()
15     fmt.Println("Padre: fin")
16 }
```

Se desea obtener la siguiente salida:

```
Padre: inicio
Hijo
Padre: fin
```

a) (5 puntos – nombre del programa: *sincrovar.go*) Empleando únicamente variables de condición sincronice padre e hijo para obtener la salida deseada. No emplee *channels* ni *sync.WaitGroup*. No se considera solución si emplea *time.Sleep()*

Sugerencia: emplee una variable global como un *flag*. Este *flag* indicará que el hijo ya terminó.

b) (3 puntos – nombre del programa: *sincrochan.go*) Empleando únicamente *channels* sincronice padre e hijo para obtener la salida deseada. No emplee *sync.WaitGroup*. No se considera solución si emplea *time.Sleep()*

2.- (8 puntos – nombre del programa: *lectescrigo*) A continuación se presenta una solución del conocido problema de los escritores y lectores. En esta solución se emplea semáforos y estos están inicializados en 1. Implemente esta solución en *Golang*, siguiendo la misma idea del algoritmo. No hay restricciones.

Writer process	Reader process
Repeat wait(mutex); writers ← writers + 1; signal(mutex); wait(resource); // write the resource wait(mutex); writers ← writers - 1; signal(mutex); signal(resource); until done	Repeat wait(mutex); if writers > 0 or readers = 0 then signal(mutex); wait(resource); wait(mutex); end-if readers ← readers + 1; signal(mutex); // read the resource wait(mutex); readers ← readers - 1; if (readers = 0) then signal(resource); end-if signal(mutex); until done

3.- (4 puntos - nombre del programa: *sincroniza.go*) Considere un programa en *Golang* que tiene $K + 1$ *gorutinas* ejecutándose en un sistema operativo similar a Linux ($K > 1$). Las primeras K *gorutinas* de este programa ejecutan una determinada tarea T1, y la *gorutina* restante ejecuta la tarea T2. La lógica del programa requiere que la tarea T1 se ejecute $N > 1$ veces, seguida de la tarea T2 ejecutada una sola vez, y este ciclo de N ejecuciones de T1 seguidas de una ejecución de T2 continúa indefinidamente. Todas las K *gorutinas* deberían poder participar en las N ejecuciones de la tarea T1, aunque no es necesario garantizar una perfecta equidad entre las *gorutinas*.

El valor N defínalo como una constante:

```
const N 10
```

Y el valor K deberá ser ingresado por teclado.

```
var K int
fmt.Println("Ingrese un valor para K")
fmt.Scanf("%d",&K)
```

Escriba un programa en *Golang* que siga las indicaciones arriba descritas. Emplee mensajes apropiados para verificar lo solicitado. No hay restricciones.

Porf. Alejandro Bello